SLE-4: Architecture Decision Record (ADR)

Course: 02AML204 – Introduction to Artificial Intelligence

PRN: 25UAM129

Name: Meet Shah

Division: B

Date: 11 October 2026

GitHub Link: https://github.com/meet-3826/first-repository

PART A: Architecture Decision Record (ADR)

Title: Selection of A* Search with Manhattan Distance over BFS for the 8-Puzzle Problem

Status: Accepted

Date: 11 October 2026

1. Context

My 8-Puzzle Solver works with a 3×3 sliding-tile board where the blank tile is moved until the goal arrangement is reached. In SLE-2, I implemented and compared Breadth-First Search (BFS) and A* Search on the same difficult starting configuration. BFS examines states level by level, while A* uses the Manhattan Distance heuristic to guide exploration. Since both algorithms found a solution, I needed to decide which one to prefer when search effort and execution time matter.

2. Decision

For the tested 8-Puzzle, I chose A* Search with Manhattan Distance as the preferred solving method. BFS remains available in the system for comparison and profiling, as shown in the SLE-3 architecture.

3. Alternatives Considered

- Alternative 1 — BFS: Implemented and profiled in SLE-2. It found the same solution depth but explored substantially more states and took longer in this experiment.
- Alternative 2 — DFS: Not implemented or profiled. It is a possible search strategy, but ordinary DFS does not guarantee a shortest solution.

4. Rationale / Justification

The SLE-2 report used the same start and goal states for both algorithms, measured three executions, and recorded states explored and solution depth. The evidence was:

Metric (SLE-2)	BFS	A*
Run 1 time (ms)	313.9576	137.5249
Run 2 time (ms)	311.8082	131.1343
Run 3 time (ms)	317.3513	125.4256
Average time (ms)	314.3724	131.3616
States explored	181,439	21,198
Solution depth	31	31

A* explored about 8.6 times fewer states and saved about 183.01 ms on average. Its f(n) = g(n) + h(n) evaluation considers both the path cost and Manhattan Distance to the goal. The py-spy flame graphs showed extra work in manhattan_distance(), yet A* still completed faster overall. In SLE-3, the Search Engine, Heuristic Module and Visited/Memory Module are separated, so the selected algorithm fits the existing modular design without

removing BFS. These measurements support preferring A* for this tested configuration; they do not prove it will always run faster.

5. Consequences

Positive:

- A* explored fewer states and required less measured runtime for the selected puzzle.
- Both algorithms reached solution depth 31, so the observed speed improvement did not increase path length.
- The independent heuristic and search components allow easier modification and comparison.

Negative / Trade-offs:

- A* performs extra Manhattan Distance calculations, visible in the profiler output.
- It must manage a priority-based search frontier and visited states, so memory use is still important.
- Actual performance depends on the puzzle state and heuristic; these timings describe one tested configuration.

6. References

SLE-2 Profiling Report (22 September 2026); SLE-3 Full C4 Architecture Model (04 October 2026); GitHub project: https://github.com/meet-3826/first-repository

PART B: Short Summary of My SLE Journey

In SLE-1, I created a simple Python chatbot and maintained a README and AI Contribution Log. In SLE-2, I implemented BFS and A* for the 8-Puzzle and compared their execution times, explored states and solution depth using repeated runs and py-spy flame graphs. A* was faster and explored fewer states in my test. In SLE-3, I prepared the Context, Container, Component and Code-level C4 diagrams to describe the solver. In SLE-4, I used that measured evidence and architecture design to document why A* with Manhattan Distance is my preferred approach for the tested puzzle.

PART C: AI Contribution Note

- AI tool used: ChatGPT.
- What AI helped with: Understanding the ADR structure, organizing the BFS–A* comparison, improving the written justification, connecting the SLE-3 C4 modules to the decision, and drafting viva practice notes. In earlier SLEs, AI assisted with code structure, profiling guidance and diagram explanations.
- What I did myself: Ran and checked the SLE-2 experiment, recorded timings and state counts, generated py-spy evidence, prepared the C4 diagrams in draw.io, and reviewed the ADR against my existing reports. I will explain the decision during the viva.

PART D: Viva Preparation Notes (Optional)

- ADR: A short record of one technical choice, its alternatives, supporting reasons and trade-offs.
- Why A*: Manhattan Distance guides search; my test showed 21,198 states for A* versus 181,439 for BFS.
- Key figures: Average time was 131.3616 ms for A* versus 314.3724 ms for BFS; both reached depth 31.
- C4 levels: Context shows user interaction; Container shows modules; Component shows search logic; Code shows functions.

- Trade-off: A* needs heuristic computation and search memory, and its improvement varies between puzzle configurations.